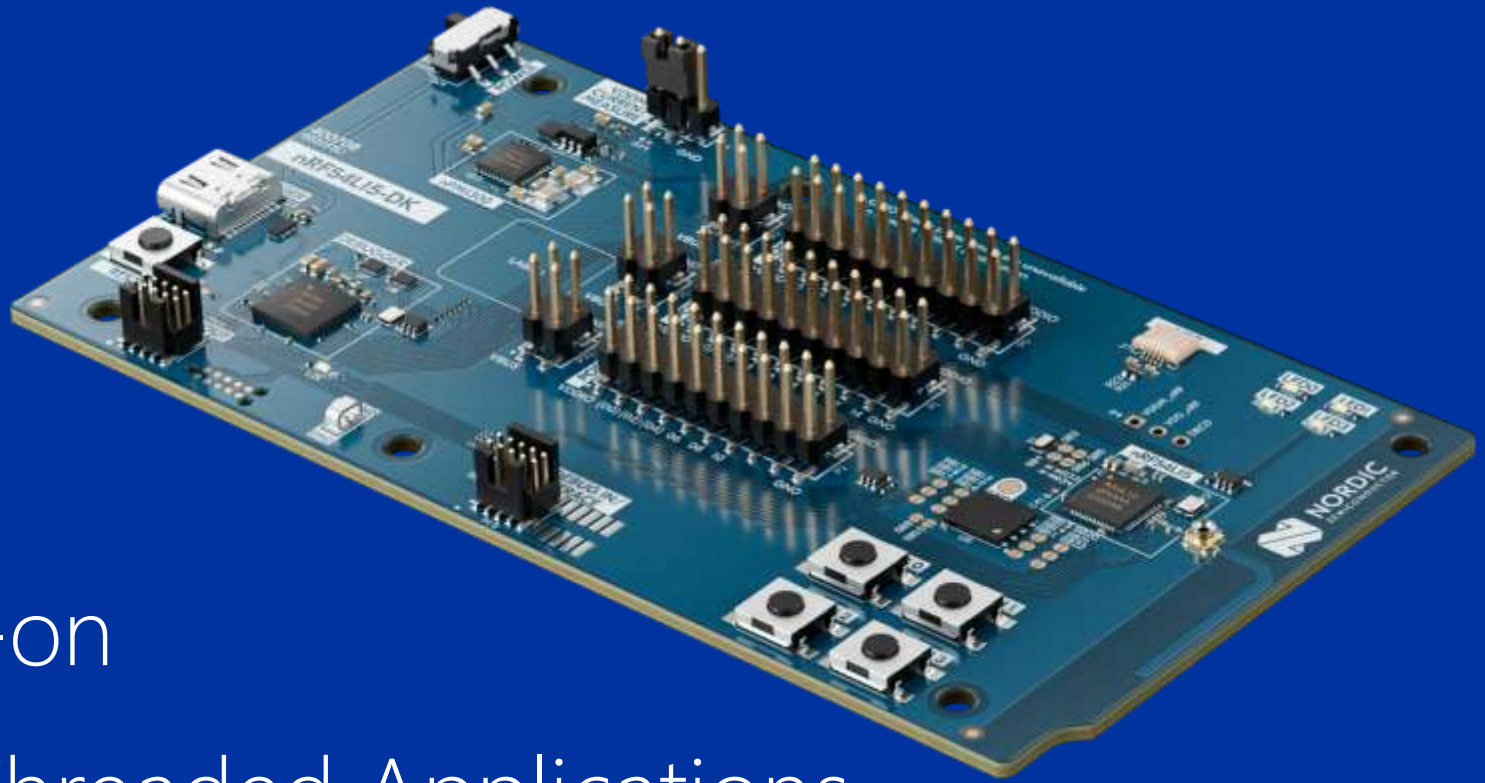


Hands-on

Multi-Threaded Applications

Copyright © 2025 Nordic Semiconductor. All rights reserved

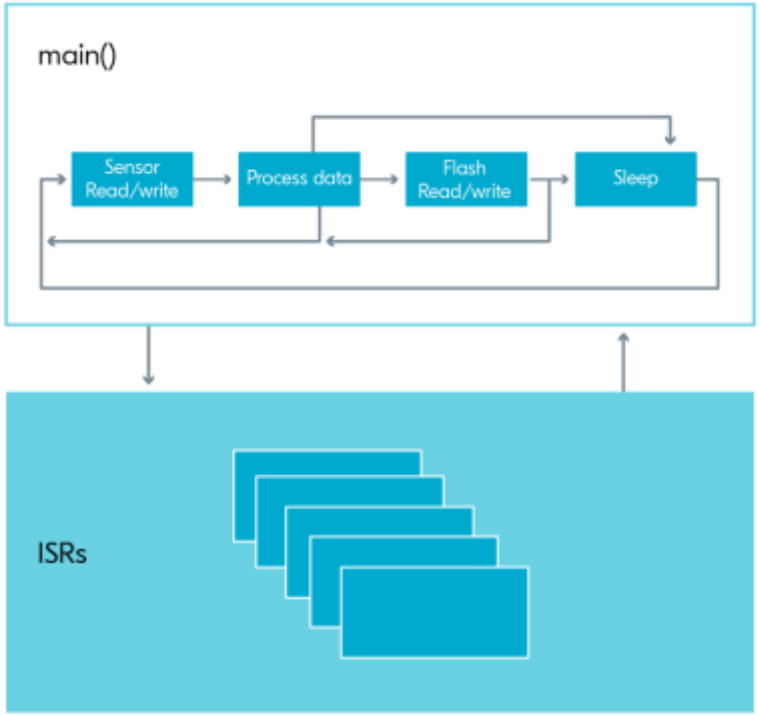
Interactive Online Learning Platform

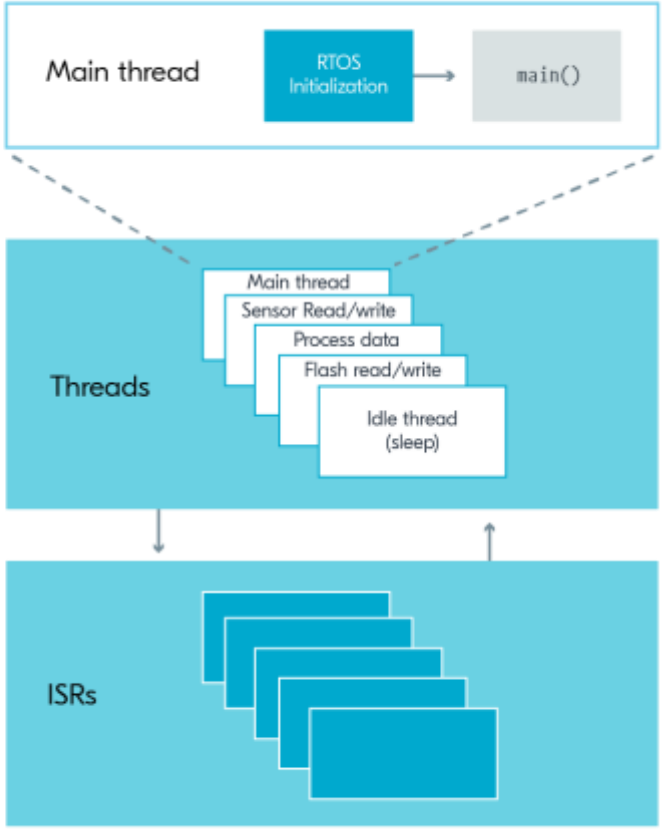
DevAcademy

A learning path from Fundamentals to Intermediate

Copyright © 2025 Nordic Semiconductor. All rights reserved

Bare-Metal vs RTOS programming





Copyright © 2025 Nordic Semiconductor. All rights reserved

© Nordic Semiconductor

Zephyr: Operation without Threads

- Thread support is not necessary in some applications:
  - Bootloaders
  - Simple event-driven applications
  - Examples intended to demonstrate core functionality

- Thread support can be disabled by setting

  `CONFIG_MULTITHREADING=n`

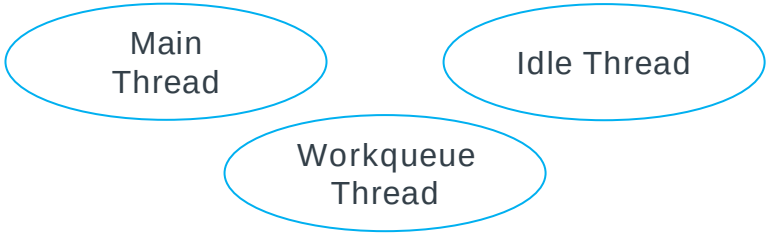
Since this configuration has a significant impact on Zephyr's functionality and testing of it has been limited, there are conditions on what can be expected to work in this configuration.

https://docs.nordicsemi.com/bundle/ncs-latest/page/zephyr/kernel/services/threads/nothread.html

© Nordic Semiconductor

Thread Types

- System Thread (aka Kernel Thread):
  a thread that the kernel spawns automatically during system initialization



- User-defined Thread:
  a kernel object that is used for application processing that is too lengthy or too complex to be performed by an ISR.

© Nordic Semiconductor

Main Thread

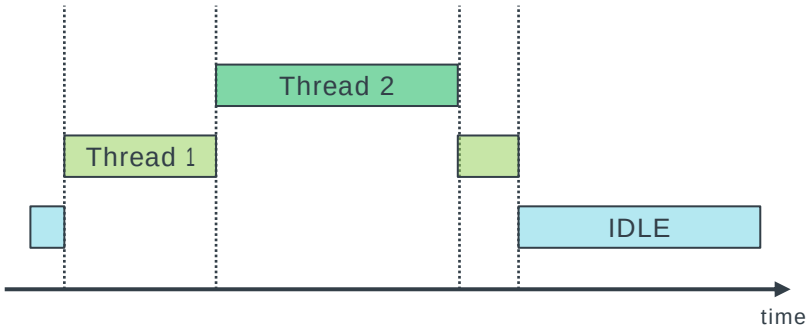
- It performs kernel initialization and then calls the application's main() function (if one is defined!)
- Default priortiy:
  Highest configured preemptible thread priority (0)
  or if preemptible thread support is disabled then lowest configured cooperative thread priority (-1)

```
int main(void)
{
    /* initialize a semaphore */
    ...

    /* register an ISR that gives the semaphore */
    ...

    /* monitor the semaphore forever */
    while (1) {
        /* wait for the semaphore to be given by the ISR */
        ...
        /* do whatever processing is now needed */
        ...
    }
}
```

© Nordic Semiconductor

Idle Thread

- Executed when there is no other work for the system to do
- If possible, it activates the board's power management support to save power.
  Otherwise, it simply performs a „do nothing" loop.
- It never terminates!
- Always lowest thread priority



© Nordic Semiconductor

Workqueue Thread

- Kernel object that uses a dedicated thread to process work items
- Workqueue must be initialized before it can be used
- Initilizing Workqueue sets its queue to empty and spawns the workqueue's thread
- Workqueue thread priority is configurable
- Regardless of workqueue thread priority the workqueue thread will yield between each submitted work item, to prevent a cooperative workqueue from starving other threads.

© Nordic Semiconductor

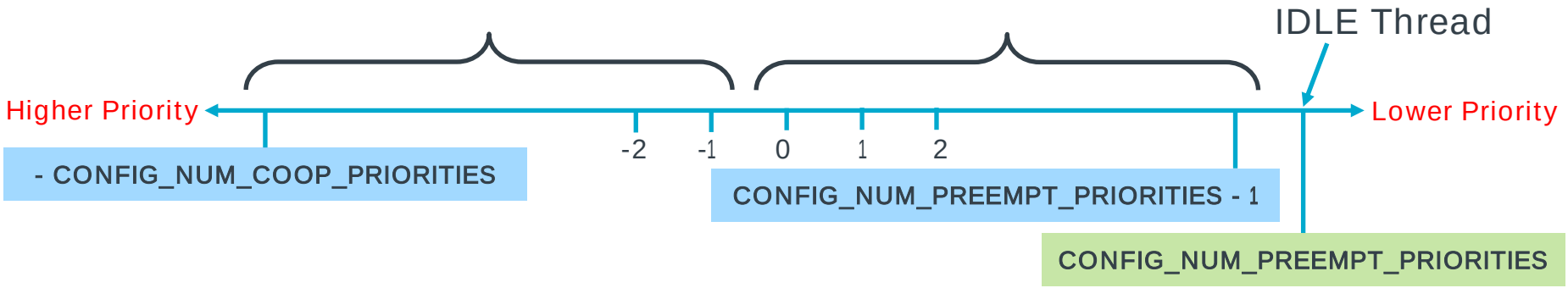
Thread Priorities

Cooperative Thread

- negative priority value.
- Once it becomes the current thread, a cooperative thread remains the current thread until it performs an action that makes it unready.

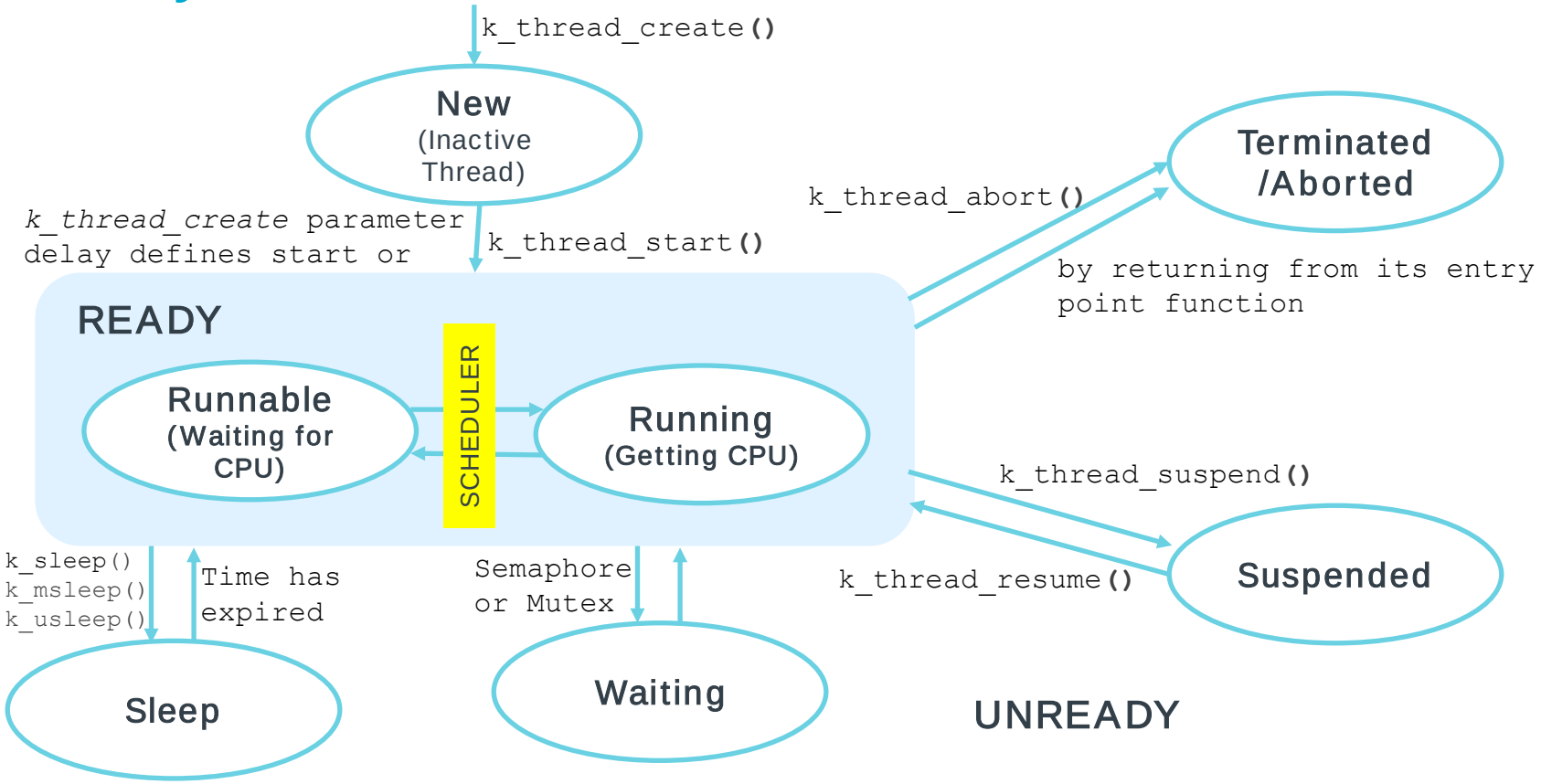
Preemptible Thread

- non-negative priority value.
- Once it becomes the current thread, a preemptible thread may be supplanted at any time if a cooperative thread, or a preemptible thread of higher or equal priority, becomes ready.



© Nordic Semiconductor

Lifecycle of a Thread



© Nordic Semiconductor

Example Code: Thread Creation

```
#define MY_STACK_SIZE 500
#define MY_PRIORITY 5

extern void my_entry_point(void *, void *, void *);

K_THREAD_STACK_DEFINE(my_stack_area, MY_STACK_SIZE);
struct k_thread my_thread_data;

k_tid_t my_tid = k_thread_create(&my_thread_data,
                    my_stack_area,
                    K_THREAD_STACK_SIZEOF(my_stack_area),
                    my_entry_point,
                    NULL, NULL, NULL,
                    MY_PRIORITY, 0, K_NO_WAIT);
```

OR

```
#define MY_STACK_SIZE 500
#define MY_PRIORITY 5

extern void my_entry_point(void *, void *, void *);

K_THREAD_DEFINE(my_tid, MY_STACK_SIZE,
                my_entry_point, NULL, NULL, NULL,
                MY_PRIORITY, 0, 0);
```

Let's try it…

https://academy.nordicsemi.com/ => Lesson 7 – Multithreaded applications

Learn through hands-on courses

Firmware Development

nRF Connect SDK Fundamentals

Learn how to develop rich portable RTOS-based applications to power your next future-proof IoT product.

9.1/10 (5612) 17391 already enrolled

Fundamentals | 8 lessons | 8-10 hours

Copyright © 2025 Nordic Semiconductor. All rights reserved

Thank you!

Copyright © 2025 Nordic Semiconductor. All rights reserved